

CAPSTONE PROJECT

USE CASE 1 – Load and Clean Chicago Crime Data

Name : Mamidoju.Vishal

Email : vishalmamidoju5@gmail.com

Objective: Build a data ingestion pipeline to read, clean, and insert crime data into Sqlite3.

Tasks:

1. Load the dataset (CSV) into a Pandas DataFrame:

- Inspect the first 10 rows; show schema and data types.
- How many columns and rows are there.

Code:

```
CSV_FILE = "data/chicago_crime_dataset.csv"
df = pd.read_csv(CSV_FILE)
# Inspect dataset
print("\n===== FIRST 10 ROWS =====")
print(df.head(10))
print("\n===== DATASET INFO =====")
df.info()
print("\n===== DATA TYPES =====")|
print(df.dtypes)
print("\n===== DATASET SHAPE =====")
print(f"Rows: {df.shape[0]}")
print(f"Columns: {df.shape[1]}")
```

Output:

```
C:\Users\chant\OneDrive\Desktop\Capstone Assessment\UseCases>py usecase1.py
```

```
===== FIRST 10 ROWS =====
   id case_number      date  ...  latitude  longitude  location
0  10000000  SS442871  03/04/2022 07:47  ...  41.716082 -87.673423  (41.71608165, -87.67342345)
1  10000001  WW348136  06/27/2015 19:34  ...  41.948512 -87.626794  (41.94851207, -87.62679436)
2  10000002  GG525832  03/31/2021 10:20  ...  41.963941 -87.629846  (41.96394084, -87.62984598)
3  10000003  NC685131  04/26/2019 13:29  ...  41.770048 -87.607711  (41.77004756, -87.60771146)
4  10000004  YT673638  09/07/2017 05:56  ...  41.727728 -87.602075  (41.72772815, -87.60207475)
5  10000005  CR348066  07/26/2017 06:02  ...  41.851770 -87.582994  (41.85177018, -87.58299423)
6  10000006  XE039726  11/05/2015 14:55  ...  41.980699 -87.522090  (41.98069888, -87.52208991)
7  10000007  QK559272  07/08/2018 05:37  ...  41.745747 -87.621489  (41.74574749, -87.62148947)
8  10000008  YD115267  01/11/2021 10:16  ...  41.852965 -87.709245  (41.85296539, -87.70924486)
9  10000009  DI878822  09/28/2018 19:40  ...  41.723562 -87.516485  (41.72356216, -87.51648472)
```

```
[10 rows x 22 columns]
```

```
===== DATASET INFO =====
<class 'pandas.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 22 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   id                    2000 non-null   int64
 1   case_number          2000 non-null   str
 2   date                 2000 non-null   str
 3   block                2000 non-null   str
 4   iucr_code            2000 non-null   int64
 5   primary_type         2000 non-null   str
 6   description          2000 non-null   str
 7   location_desc        1906 non-null   str
 8   arrest               2000 non-null   bool
 9   domestic             2000 non-null   bool
10   beat_num             2000 non-null   int64
11   district_code        2000 non-null   int64
12   ward_no              1963 non-null   float64
13   community_code       1964 non-null   float64
14   fbi_code             2000 non-null   str
15   x_coordinate         1933 non-null   float64
16   y_coordinate         1936 non-null   float64
17   year                 2000 non-null   int64
18   date_of_update       2000 non-null   str
19   latitude             1927 non-null   float64
20   longitude            1936 non-null   float64
21   location             1926 non-null   str
dtypes: bool(2), float64(6), int64(5), str(9)
memory usage: 316.5 KB
```

```
===== DATA TYPES =====
id                    int64
case_number          str
date                 str
block                str
iucr_code            int64
primary_type         str
description          str
location_desc        str
arrest               bool
domestic             bool
beat_num             int64
district_code        int64
ward_no              float64
community_code       float64
fbi_code             str
x_coordinate         float64
y_coordinate         float64
year                 int64
date_of_update       str
latitude             float64
longitude            float64
location             str
dtype: object
```

2. Clean the dataset:

- Convert the Date column into datetime format.
- Identify and handle missing values for key fields.
- Standardize categorical fields (strip whitespace, unify case).

Code:

```
# Convert date column
df["date"] = pd.to_datetime(df["date"], errors="coerce")
print("\nDate column converted to datetime.")
# Check missing values
print("\n===== MISSING VALUES BEFORE CLEANING =====")
print(df.isnull().sum())
# Calculate missing percentages
missing_percentage = np.round((df.isnull().sum().to_numpy() / len(df)) * 100, 2)

missing_df = pd.DataFrame({
    "column_name": df.columns,
    "missing_percentage": missing_percentage
})

print("\n===== MISSING VALUE PERCENTAGE =====")
print(missing_df)

# Drop columns with over 50% missing values
columns_to_drop = missing_df[
    missing_df["missing_percentage"] > 50
]["column_name"].tolist()

if columns_to_drop:
    print("\nDropping columns:", columns_to_drop)
    df.drop(columns=columns_to_drop, inplace=True)
else:
    print("\nNo columns have more than 50% missing values.")

# Fill missing numeric values with median
numeric_columns = df.select_dtypes(include=np.number).columns

for column in numeric_columns:
    if df[column].isnull().sum() > 0:
        df[column] = df[column].fillna(df[column].median())

# Fill missing text values with Unknown
categorical_columns = df.select_dtypes(
    include=["object", "string"]
).columns

for column in categorical_columns:
    if df[column].isnull().sum() > 0:
        df[column] = df[column].fillna("Unknown")

# Standardize text columns
for column in categorical_columns:
    df[column] = df[column].astype(str).str.strip().str.upper()
```

Output:

```
Date column converted to datetime.

===== MISSING VALUES BEFORE CLEANING =====
id                0
case_number       0
date              0
block             0
iucr_code         0
primary_type      0
description       0
location_desc     94
arrest           0
domestic         0
beat_num         0
district_code     0
ward_no          37
community_code    36
fbi_code         0
x_coordinate      67
y_coordinate      64
year             0
date_of_update    0
latitude         73
longitude        64
location         74
dtype: int64
```

3. Generate new features:

- Extract Year, Month, and DayOfWeek from the Date field in chicago_crime_dataset.csv.

Code:

```
# Create date-based features
df["crime_year"] = df["date"].dt.year
df["crime_month"] = df["date"].dt.month
df["day_of_week"] = df["date"].dt.day_name()

# Display cleaned data
print("\n===== CLEANED DATA =====")
print(df.head(10))

print("\n===== MISSING VALUES AFTER CLEANING =====")
print(df.isnull().sum())
```

Output:

```
===== CLEANED DATA =====
   id case_number      date  ... crime_year  crime_month day_of_week
0  10000000  SS442871 2022-03-04 07:47:00  ...    2022         3    Friday
1  10000001  WW348136 2015-06-27 19:34:00  ...    2015         6    Saturday
2  10000002  GG525832 2021-03-31 10:20:00  ...    2021         3   Wednesday
3  10000003  NC685131 2019-04-26 13:29:00  ...    2019         4    Friday
4  10000004  YT673638 2017-09-07 05:56:00  ...    2017         9   Thursday
5  10000005  CR348066 2017-07-26 06:02:00  ...    2017         7   Wednesday
6  10000006  XE039726 2015-11-05 14:55:00  ...    2015        11   Thursday
7  10000007  QK559272 2018-07-08 05:37:00  ...    2018         7     Sunday
8  10000008  YD115267 2021-01-11 10:16:00  ...    2021         1    Monday
9  10000009  DI878822 2018-09-28 19:40:00  ...    2018         9    Friday

[10 rows x 25 columns]
```

4. Use NumPy:

- Calculate the percentage of missing values per column.
- Can you drop any columns with > 50% missing?

Code:

```
# Fill missing numeric values with median
numeric_columns = df.select_dtypes(include=np.number).columns

for column in numeric_columns:
    if df[column].isnull().sum() > 0:
        df[column] = df[column].fillna(df[column].median())

# Fill missing text values with Unknown
categorical_columns = df.select_dtypes(
    include=["object", "string"]
).columns

for column in categorical_columns:
    if df[column].isnull().sum() > 0:
        df[column] = df[column].fillna("Unknown")
```

Output:

```
===== MISSING VALUE PERCENTAGE =====
      column_name  missing_percentage
0             id                0.00
1      case_number                0.00
2             date                0.00
3             block                0.00
4          iucr_code                0.00
5      primary_type                0.00
6      description                0.00
7  location_desc                4.70
8          arrest                0.00
9      domestic                0.00
10      beat_num                0.00
11  district_code                0.00
12          ward_no                1.85
13  community_code                1.80
14          fbi_code                0.00
15  x_coordinate                3.35
16  y_coordinate                3.20
17          year                0.00
18  date_of_update                0.00
19      latitude                3.65
20      longitude                3.20
21      location                3.70

No columns have more than 50% missing values.
```

5. Insert cleaned data into MySQL or Sqlite3:

- Design a crime table schema in MySQL or Sqlite3.

```
# Save cleaned data to SQLite
DATABASE_FILE = "chicago_crime.db"
connection = sqlite3.connect(DATABASE_FILE)

try:
    df.to_sql(
        "chicago_crimes",
        connection,
        if_exists="replace",
        index=False
    )
    print("\nData successfully inserted into SQLite database.")
finally:
    connection.close()

# Final output
print("\n=====")
print("USE CASE 1 COMPLETED SUCCESSFULLY")
print("=====")
print(f"Database file: {DATABASE_FILE}")
print("Table name: chicago_crimes")
print(f"Final rows: {df.shape[0]}")
print(f"Final columns: {df.shape[1]}")
print("=====")
```

Output:

```
Data successfully inserted into SQLite database.

=====
USE CASE 1 COMPLETED SUCCESSFULLY
=====
Database file: chicago_crime.db
Table name: chicago_crimes
Final rows: 2000
Final columns: 25
=====
```

Insight

- The Chicago crime dataset was successfully loaded using **Pandas**.
- The dataset structure, data types, number of rows, and columns were examined.
- Missing values were identified and calculated as percentages.
- Columns with more than **50% missing values** were removed.
- Missing numerical values were replaced using the **median**.
- Missing categorical values were replaced with "**Unknown**".
- Text data was standardized by removing extra spaces and converting it to uppercase.
- The **date** column was converted into datetime format.
- New features such as **crime year, crime month, and day of the week** were created for further analysis.